```
        RequireAuthenticatedUser().Build();
});
```

The `FallbackPolicy` becomes the default authorization policy for the application wherever no other authorization policies are specified, such as through an attribute or a convention.

You still need users to be able to reach the login page to authenticate. For those instances, you can use the `AllowAnonymousAttribute`. Just like the `AuthorizeAttribute`, you can apply this to the `PageModel` class of the target endpoint:

```
[AllowAnonymous]
public class LoginModel : PageModel
```

Alternatively, you can combine the `AuthorizeFolder` method's application with the `AllowAnonymousToPage` method, which overrides the authorization for the specified page, allowing users to access it without having to be authenticated.

Listing 10.7 Allowing anonymous access to individual endpoints

```
builder.Services.AddRazorPages(options => {
    options.Conventions.AuthorizeFolder("/");              ❶
    options.Conventions.AllowAnonymousToPage("/Login");    ❷
});
```

❶ Prevent anonymous access to the application by default.

❷ Allow anonymous access to the Login page in the protected folder.

So far we have looked at authorization that depends solely on whether the user is authenticated or not. In the real world, anything more than the simplest application will require finer-grained access controls. In the rest of the chapter, we will explore strategies to express those requirements and apply them to users.